

Mão na massa: Rede Neural prevendo desempenho no trabalho

Novo cenário, novo dataset: 600 registros, prevendo se uma pessoa teve BOM DESEMPENHO numa tarefa, a partir de horas de sono e nível de estresse.

Existe até um nome pra esse fenômeno: a Lei de Yerkes-Dodson -- estresse baixo demais ou alto demais atrapalha; um nível moderado maximiza o desempenho. Um padrão em 'círculo', não uma reta.

Vamos treinar Regressão Logística (baseline) e Rede Neural, no mesmo dataset, e comparar de verdade.



Importações necessárias

Regressão Logística (baseline) + MLPClassifier (rede neural) + StandardScaler (normalização, igual no KNN).

LEMBRETE DE SINTAXE

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score
```



Carregar e explorar os dados

Confiram o head() e procurem por valores nulos antes de seguir.

LEMBRETE DE SINTAXE

```
df = pd.read_csv("desempenho_alunos.csv")  
print(df.head())  
print(df.isna().sum())
```

Tratar valores nulos

Tem sujeira de propósito nos dados -- 15 valores nulos em `nivel_estresse`.

LEMBRETE DE SINTAXE

```
df = df.dropna(subset=["nivel_estresse"])
```



Definir features (X) e alvo (y)

As 2 entradas, e o alvo binário (bom_desempenho).

LEMBRETE DE SINTAXE

```
X = df[["horas_sono", "nivel_estresse"]]  
y = df["bom_desempenho"]
```

Separar treino e teste

stratify=y de novo, igual sempre fazemos em classificação.

LEMBRETE DE SINTAXE

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=5, stratify=y  
)
```



Normalizar com StandardScaler

Essencial pra rede neural, igual já era essencial pro KNN.

LEMBRETE DE SINTAXE

```
scaler = StandardScaler()  
X_train_norm = scaler.fit_transform(X_train)  
X_test_norm = scaler.transform(X_test)
```



Treinar a Regressão Logística (baseline)

Antes da rede, sempre vale ter um baseline simples pra comparar.

LEMBRETE DE SINTAXE

```
log_reg = LogisticRegression()
log_reg.fit(X_train_norm, y_train)

pred_log = log_reg.predict(X_test_norm)
print(f"Acuracia Regressao Logistica: {accuracy_score(y_test, pred_log):.2%}")
```



Treinar a Rede Neural

hidden_layer_sizes=(6,) significa 1 camada oculta, com 6 neurônios.

LEMBRETE DE SINTAXE

```
mlp = MLPClassifier(hidden_layer_sizes=(6,), activation="relu",
                    max_iter=8000, random_state=42)
mlp.fit(X_train_norm, y_train)

pred_mlp = mlp.predict(X_test_norm)
print(f"Acuracia Rede Neural: {accuracy_score(y_test, pred_mlp):.2%}")
```

Testar diferentes arquiteturas

Igual testamos K, max_depth, n_estimators -- aqui é a quantidade de neurônios.

LEMBRETE DE SINTAXE

```
arquiteturas = [(2,), (4,), (6,), (8,), (12,), (8,4), (16,8)]
acuracias = []
for arq in arquiteturas:
    m = MLPClassifier(hidden_layer_sizes=arq, activation="relu",
                      max_iter=8000, random_state=42)
    m.fit(X_train_norm, y_train)
    acuracias.append(accuracy_score(y_test, m.predict(X_test_norm)))

plt.plot(range(len(arquiteturas)), acuracias, marker="o")
plt.show()
```



PASSO 10 de 10

Avaliação final

Rodem no desempenho_final.csv -- comparando os dois modelos de novo.

LEMBRETE DE SINTAXE

```
df_final = pd.read_csv("desempenho_final.csv").dropna(subset=["nivel_estresse"])
X_final = df_final[["horas_sono", "nivel_estresse"]]
y_final = df_final["bom_desempenho"]
X_final_norm = scaler.transform(X_final)

pred_final_mlp = mlp.predict(X_final_norm)
pred_final_log = log_reg.predict(X_final_norm)
print("Rede Neural:", accuracy_score(y_final, pred_final_mlp))
print("Reg. Logística:", accuracy_score(y_final, pred_final_log))
```



REFLETIR

Comparando com tudo que já vimos

A Rede Neural bateu a Regressão Logística por uma margem enorme aqui (~62% vs. ~86-88%). Comparando com os outros 4 algoritmos que já testamos (KNN, Árvore, Random Forest): esse ganho é parecido, maior, ou menor? Por que será?

Dica: o ganho da rede é maior quando o padrão real tem uma curva/formato que uma reta não capta.



5 algoritmos de classificação, todos praticados

Regressão Logística, KNN, Árvore de Decisão, Random Forest e agora Rede Neural -- todos treinados, avaliados, comparados. A diferença entre eles não é acidente: cada um tem pontos fortes diferentes.

A seguir: Keras -- a mesma rede, escrita numa ferramenta usada pra imagem, texto e muito mais.

